Lezione 3 — Requirements Engineering: dal dominio a requisiti verificabili

Percorso intensivo di AI Software Engineering

Durata indicativa: 30–45 minuti

Posizione nel percorso: Domain → REQUIREMENTS → Spec → Test → Agent → Verify

Obiettivo della lezione

Trasformare ciò che abbiamo imparato durante il Domain Discovery in requisiti abbastanza precisi da poter essere verificati, senza anticipare inutilmente il design della soluzione.

1. Dal Domain Discovery ai Requirements

Nella Lezione 2 abbiamo lavorato sul problema: eventi, linguaggio, regole, esempi, confini e domande aperte. Ora facciamo un passaggio diverso: non stiamo più chiedendo “come funziona il dominio?”, ma “che cosa deve essere vero del sistema perché il problema sia considerato risolto?”.

```
Domain Discovery
    ↓
termini, eventi, regole, esempi, UNKNOWN
    ↓
Requirements Engineering
    ↓
obblighi verificabili del sistema
    ↓
Specification
    ↓
soluzione abbastanza precisa da essere costruita
```

La distinzione è decisiva per il software engineering con AI: se saltiamo direttamente dal dominio alla soluzione, l’agente tende a colmare le lacune con scelte plausibili ma non autorizzate.

> **Principio guida**
>
> Un requisito descrive ciò che deve essere vero o osservabile. Una specifica di soluzione descrive abbastanza del “come” da rendere il lavoro implementabile. Le due cose sono collegate, ma non sono la stessa cosa.

2. Problema, requisito e design: tre livelli diversi

Prendiamo il dominio del ticket di manutenzione.

Livello	Domanda	Esempio
Problema / bisogno	Perché stiamo intervenendo?	Il gestore non deve chiudere ticket mentre esistono attività operative ancora aperte.
Requisito	Che cosa deve garantire il sistema?	Il sistema deve impedire la chiusura di un ticket se esiste almeno un intervento aperto.
Design / soluzione	Come lo realizziamo?	Un servizio applicativo controlla gli interventi e rifiuta il comando CloseTicket.

Il salto pericoloso è questo:

```
“Serve impedire la chiusura”  →  “aggiungiamo un campo tickets.can_close”
```

Il primo è un vincolo di dominio/prodotto. Il secondo è già una decisione di design. Potrebbe essere corretta, ma non è implicata dal problema.

3. Che cos’è Requirements Engineering

Requirements Engineering non significa semplicemente “scrivere una lista di feature”. È l’insieme delle attività con cui si elicita, analizza, documenta, valida e gestisce ciò che stakeholder e sistema richiedono durante il ciclo di vita. ISO/IEC/IEEE 29148 tratta esplicitamente processi e prodotti della requirements engineering e descrive il costrutto di un buon requisito, oltre alla natura iterativa e ricorsiva del processo.

Mental model

Il requisito è un contratto di significato prima di essere un contratto tecnico: deve poter essere discusso con il dominio e poi verificato dal team.

4. Una tassonomia pratica dei requisiti

Tipo	Cosa esprime	Esempio
Business goal	Risultato organizzativo desiderato	Ridurre le chiusure errate dei ticket.
Stakeholder/User requirement	Bisogno espresso dal punto di vista di un attore	Il gestore deve poter capire perché un ticket non è chiudibile.
Functional requirement	Comportamento richiesto al sistema	Il sistema deve rifiutare la chiusura se esistono interventi aperti.
Quality/NFR	Qualità misurabile del comportamento	Il controllo di chiusura deve completarsi entro 500 ms nel 95° percentile.
Constraint	Vincolo esterno o tecnologico realmente imposto	L’audit delle chiusure deve essere conservato per il periodo previsto dalla policy applicabile.
Invariant	Condizione che non deve mai essere violata	Un ticket chiuso non può avere interventi in stato operativo aperto.

Queste categorie servono a non mescolare motivazioni, comportamento, qualità e vincoli. In molti progetti la stessa frase contiene tutto insieme e diventa difficile da testare o cambiare.

5. Come riconoscere un requisito utile

Un requisito utile dovrebbe superare almeno questa checklist operativa:

- Necessario: se lo elimino, sto perdendo un bisogno reale?
- Univoco: due persone ragionevoli lo interpretano nello stesso modo?
- Verificabile: posso costruire una prova che dica pass/fail?
- Atomico: contiene una sola obbligazione principale?
- Fattibile: è realizzabile con i vincoli noti?
- Tracciabile: so da quale regola, bisogno, rischio o decisione proviene?
- Quanto possibile implementation-free: dice che cosa deve accadere senza imporre prematuramente componenti, database o framework?

Test mentale

Se per capire se il requisito è soddisfatto devo chiedere “cosa intendi esattamente?”, il requisito non

è ancora pronto.

6. Invarianti: requisiti ad alta densità

Un'invariante è una regola che deve rimanere vera in tutti gli stati rilevanti del sistema. È particolarmente importante perché comprime molta conoscenza del dominio in una frase verificabile.

```
INVARIANTE
Un Ticket CHIUSO non può avere alcun Intervento in stato PIANIFICATO, IN_CORSO o
IN_ATTESA_DI_ESITO.
```

Da questa singola invariante derivano molte conseguenze:

- il comando di chiusura deve essere rifiutato quando l'invariante verrebbe violata;
- importazioni o job asincroni non devono poter creare uno stato illegale;
- eventuali procedure di migrazione devono preservare l'invariante;
- i test dovrebbero includere sia casi nominali sia combinazioni di stati.

Un'invariante non è necessariamente una colonna, una constraint SQL o una classe. Quelle sono possibili strategie di enforcement, cioè design.

7. EARS: dare struttura al linguaggio naturale

EARS (Easy Approach to Requirements Syntax), sviluppato da Alistair Mavin e colleghi, "vincola gentilmente" il linguaggio naturale usando pochi pattern e un ordine stabile delle clausole. L'obiettivo è ridurre ambiguità e vaghezza senza imporre un linguaggio formale complesso.

```
Forma generica EARS:
While <precondizione>, when <trigger>, the <system> shall <risposta>
```

In italiano possiamo mantenere il concetto anche se la sintassi canonica è inglese:

Pattern	Forma	Esempio sul ticket
Ubiquitous	Il sistema deve …	Il sistema deve registrare l'autore e l'istante di ogni chiusura.
State-driven	Mentre <stato>, il sistema deve …	Mentre esiste un intervento aperto, il sistema deve considerare il ticket non chiudibile.
Event-driven	Quando <evento>, il sistema deve …	Quando un gestore richiede la chiusura, il sistema deve verificare l'assenza di interventi aperti.
Unwanted behaviour	Se <situazione indesiderata>, allora …	Se la chiusura violerebbe l'invariante, allora il sistema deve rifiutare la richiesta e comunicare il motivo.
Optional feature	Dove/Se è presente <feature>, il sistema deve …	Se è attivo il controllo qualità, il sistema deve richiedere anche un esito verificato.

Che problema risolve EARS?

Trasforma frasi come "il sistema dovrebbe gestire bene la chiusura" in obblighi più strutturati, esplicitando stato, trigger e risposta. Non sostituisce il Domain Discovery: struttura ciò che è già

stato compreso.

8. Quando usare EARS e quando no

Tecnica	Usala quando	Preferisci altro quando
EARS	Serve una frase testabile con stato/trigger/risposta	Hai molte combinazioni: meglio decision table o state machine.
User story	Vuoi mantenere visibile attore, bisogno e valore	Ti serve precisione normativa o comportamentale.
Given/When/Then	Vuoi esempi concreti osservabili, spesso vicini ai test	Stai ancora cercando la regola generale.
Decision table	Molte condizioni combinatorie producono esiti diversi	Il comportamento è lineare e semplice.
State diagram	La correttezza dipende soprattutto da transizioni di stato	Il problema è una singola regola locale.

La qualità nasce dalla scelta della rappresentazione più adatta, non dal forzare tutto in un solo formato.

9. Acceptance criteria: il ponte verso la verifica

Un requisito dice che cosa deve essere vero. Gli acceptance criteria aiutano a stabilire come riconosciamo che lo è. Non sono necessariamente test automatici, ma devono essere abbastanza concreti da poter diventare test.

```
Regola: un ticket non può essere chiuso con interventi aperti.

Esempio 1
Given un ticket con un intervento PIANIFICATO
When il gestore richiede la chiusura
Then la chiusura viene rifiutata
And viene indicato che esiste un intervento aperto

Esempio 2
Given un ticket i cui interventi sono tutti COMPLETATI
When il gestore richiede la chiusura
Then il ticket viene chiuso
```

La documentazione Cucumber descrive Given come contesto iniziale, When come evento/azione e Then come risultato atteso osservabile. Il punto importante è “osservabile”: evitare di trasformare gli scenari di dominio in asserzioni su dettagli interni come tabelle o metodi privati.

10. Regole generali ed esempi concreti devono convivere

Solo regole generali → rischio di interpretazioni divergenti. Solo esempi → rischio di overfitting ai casi mostrati. Una buona specificazione comportamentale usa entrambi.

```
RULE
Il ticket è chiudibile solo se non esistono interventi operativamente aperti.

EXAMPLES
```

```
[COMPLETATO] → chiudibile
[ANNULLATO] → chiudibile
[COMPLETATO, PIANIFICATO] → non chiudibile
[] → ?  ← domanda di dominio, non supposizione
```

Qui torna Example Mapping

La tecnica studiata nella Lezione 2 aiuta proprio a tenere insieme Rule, Examples e Questions. Requirements Engineering usa quel materiale per decidere cosa è abbastanza definito da diventare un obbligo verificabile.

11. Non-Functional Requirements: smettere di scrivere “veloce”, “sicuro”, “scalabile”

I requisiti di qualità sono spesso quelli che un agente AI “abbellisce” con aggettivi vaghi. “La pagina deve essere veloce” non è un requisito verificabile.

```
VAGO
La chiusura del ticket deve essere veloce.

MISURABILE
In condizioni operative normali, il 95% delle richieste di chiusura deve produrre una
risposta entro 500 ms, escluso il tempo di rete del client.
```

La misura corretta dipende dal contesto. Non inventare numeri perché “sembrano ragionevoli”: se 500 ms non provengono da un bisogno, da dati o da una decisione, sono un ASSUMED.

- Performance: latenza, throughput, volumi, finestra di osservazione.
- Reliability: disponibilità, error budget, recupero, perdita dati tollerabile.
- Security: chi può fare cosa, condizioni di autorizzazione, audit, protezione dati.
- Usability/accessibility: task completion, standard applicabili, supporto input/assistive technologies.
- Operability: osservabilità, diagnosi, rollback, configurazione.

12. Traceability: la catena che rende l’AI controllabile

Per un team umano la tracciabilità è utile. Per un flusso agentico diventa una forma di controllo: ogni decisione implementata dovrebbe poter risalire a una fonte autorizzata.

```
Domain evidence
   ↓
REQ-17  “impedire la chiusura con interventi aperti”
   ↓
AC-17.1 / AC-17.2
   ↓
Test di accettazione / property test
   ↓
Seam applicativa CloseTicket
   ↓
Implementazione
```

Se durante l'implementazione compare una regola senza un percorso di tracciabilità, abbiamo un segnale di allarme: forse l'agente ha introdotto una decisione non autorizzata.

13. UNKNOWN non è un requisito incompleto da "finire" con l'AI

```
UNKNOWN
Un ticket senza alcun intervento può essere chiuso?
```

Un LLM potrebbe completare: "sì, perché non ci sono interventi aperti". È logicamente plausibile, ma potrebbe essere business logic inventata. Finché non abbiamo una fonte, deve restare UNKNOWN oppure essere trasformata in una decisione esplicita.

Regola operativa

ASSUMED e UNKNOWN non entrano in una spec agent-ready come se fossero fatti. Devono essere risolti, esplicitamente marcati come assunzioni accettate oppure lasciati fuori scope.

14. Quattro failure mode tipici con gli agenti

Failure mode	Cosa succede	Contromisura
Requirement laundering	Un'ipotesi viene riscritta con tono autorevole e sembra un requisito	Etichettare KNOWN/ASSUMED/UNKNOWN/DECIDED e mantenere la fonte.
Premature design	Il requisito contiene DB, endpoint, queue o pattern non necessari	Riscrivere in termini di comportamento osservabile e vincoli reali.
Synthetic completeness	L'agente riempie ogni sezione della spec anche senza evidenza	Accettare sezioni vuote o UNKNOWN; completezza apparente ≠ correttezza.
Test mirroring	I test ripetono l'implementazione invece di verificare il requisito	Testare all'esterno, alla seam più alta utile, usando esempi e invarianti.

15. Dove siamo nel workflow di Matt Pocock

La documentazione corrente delle skill mantiene la catena principale:

```
grill-with-docs → to-spec → to-tickets → implement → code-review
```

`grill-with-docs` usa il meccanismo di grilling e la disciplina `domain-modeling` per chiarire decisioni e linguaggio. `to-spec` non dovrebbe riaprire l'intervista: sintetizza ciò che è già stato deciso e, nella versione corrente, propone anche le seam di test prima di pubblicare la spec.

Collegamento con questa lezione

Requirements Engineering è il lavoro concettuale che rende possibile un buon passaggio da grill/domain-modeling a to-spec. `to-spec` è una skill di sintesi, non un sostituto dell'analisi dei requisiti.

In pratica:

```
Domain Discovery
  ↳ EventStorming / Domain Storytelling / Example Mapping
  ↳ grill-with-docs + domain-modeling
        ↓
Requirements Engineering
  ↳ invarianti, requisiti, EARS, acceptance criteria, NFR, traceability
        ↓
/to-spec
        ↓
Spec agent-ready
```

16. Classificazione degli strumenti introdotti oggi

Elemento	Cos'è	Problema risolto	Alternative
EARS	Tecnica / controlled natural language	Ambiguità nel testo dei requisiti	Use cases, formal methods, tabelle decisionali
User Story	Formato di comunicazione agile	Mantiene visibili attore, bisogno e valore	Job story, use case, outcome statement
Given/When/Then	Notazione per esempi comportamentali	Rende concreto l'esito osservabile	EARS, esempi tabellari, test case tradizionali
Decision Table	Tecnica di analisi/spec	Gestisce combinazioni di condizioni	State machine, truth table, regole separate
to-spec	Skill/workflow agentico	Conserva decisioni mature in una spec persistente	PRD manuale, design doc, issue/spec template

17. Esercizio pratico — 25 minuti

Riprendi la capability che hai usato nella Lezione 2 oppure usa "chiusura ticket". L'obiettivo non è produrre molte frasi: è creare una piccola catena di requisiti che potresti consegnare a un altro sviluppatore o a un agente senza chiedergli di inventare il dominio.

0–5 min — Fonte: Scrivi 5 elementi di evidenza dal discovery: 2 regole, 2 esempi e 1 UNKNOWN.

5–10 min — Requisiti: Trasforma solo ciò che è abbastanza deciso in 3–5 requisiti con ID (REQ-1, REQ-2…). Evita dettagli di implementazione.

10–15 min — EARS: Riscrivi almeno 2 requisiti in pattern EARS: uno event-driven e uno unwanted-behaviour.

15–20 min — Acceptance: Per un requisito scrivi 2 esempi Given/When/Then, includendo almeno un caso negativo.

20–23 min — NFR: Aggiungi un requisito di qualità. Se non hai una soglia giustificata, lascia il valore come UNKNOWN invece di inventarlo.

23–25 min — Traceability: Disegna la catena Evidence → Requirement → Example/Test e verifica che nessuna riga provenga solo da "sembra ragionevole".

Criterio di successo

> Un altro ingegnere deve poter capire che cosa è obbligatorio, perché lo è e come osservarne la correttezza, senza dover dedurre business logic non dichiarata.

18. Takeaway

```
Domain
  ↓  “cosa succede e perché?”
Requirements
  ↓  “che cosa deve essere vero?”
Spec
  ↓  “qual è la soluzione buildable?”
Test
  ↓  “come dimostriamo che è vero?”
Agent
  ↓
Verify
```

- Un requisito non è una feature description e non è ancora un design.
- Invarianti e acceptance criteria rendono il comportamento verificabile da prospettive complementari.
- EARS è una tecnica leggera per ridurre l’ambiguità del linguaggio naturale, non una metodologia completa.
- Gli NFR devono essere misurabili o esplicitamente ancora UNKNOWN.
- La traceability è una difesa contro decisioni “inventate” dagli agenti.
- `to-spec` funziona meglio quando le decisioni sono già mature: la spec deve catturare, non fabbricare, il consenso.

La prossima lezione naturale è: Lezione 4 — “Spec-Driven Development: trasformare i requisiti in una specifica agent-ready”. Qui separeremo con precisione Requirements, Spec e Design, e confronteremo workflow manuali con Spec Kit, OpenSpec, Kiro, Tessl e il ruolo di `to-spec`.

Fonti principali

- IEEE Standards Association — ISO/IEC/IEEE 29148-2018, Systems and software engineering — Life cycle processes — Requirements engineering. https://standards.ieee.org/ieee/29148/6937/
- Alistair Mavin — EARS: Easy Approach to Requirements Syntax (sito ufficiale). https://alistairmavin.com/ears/
- Cucumber — Example Mapping. https://cucumber.io/docs/bdd/example-mapping/
- Cucumber — Gherkin Reference. https://cucumber.io/docs/gherkin/reference/
- Matt Pocock — skills repository: domain-modeling, grill-with-docs, to-spec. https://github.com/mattpocock/skills